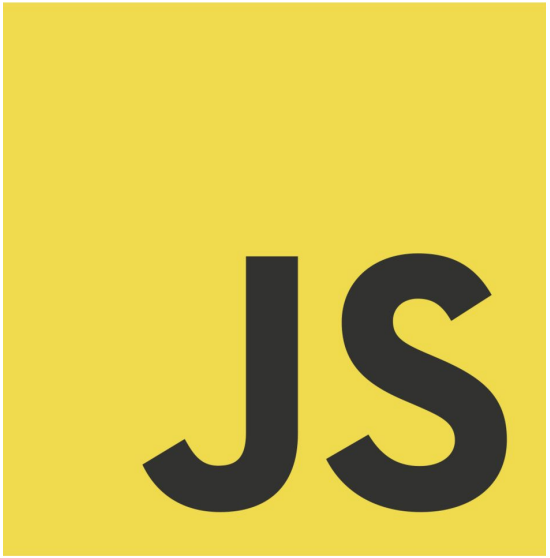
Atualizado em 08.10.2026
Versão 0.0.1

JavaScript

Funções, Closures e Módulos



JavaScript: Funções, Closures e Módulos © 2026 por Allyson Souza está licenciado sob a **CC BY-NC 4.0**. Para ver uma cópia da licença, visite https://creativecommons.org/licenses/by-nc/4.0/

Funções

declaração, propriedades e manipulação

Três formas de criação de funções

1. **Declaração de funções** (ou funções nomeadas)
2. **Expressão de funções** (atribuindo à variáveis, constantes e propriedades de objetos)
3. **Funções seta** (*arrow functions*)

Características de funções

Em JavaScript, funções são objetos de primeira classe, isso significa:

- Podem ser **atribuídas a variáveis**
- Podem ser **passadas como argumento**
- Podem ser **retornadas** a partir da execução de uma outra função

Expressão de função

function expression

Expressão de função

Uma **expressão de função** (*function expression*) é muito similar e tem quase a mesma sintaxe de uma declaração de função (*function statement*).

A principal diferença entre uma expressão de função e a declaração de uma função é o **nome da função**, o qual pode ser omitido em expressões de funções para criar funções anônimas (funções que não tem nome).

Expressão de função

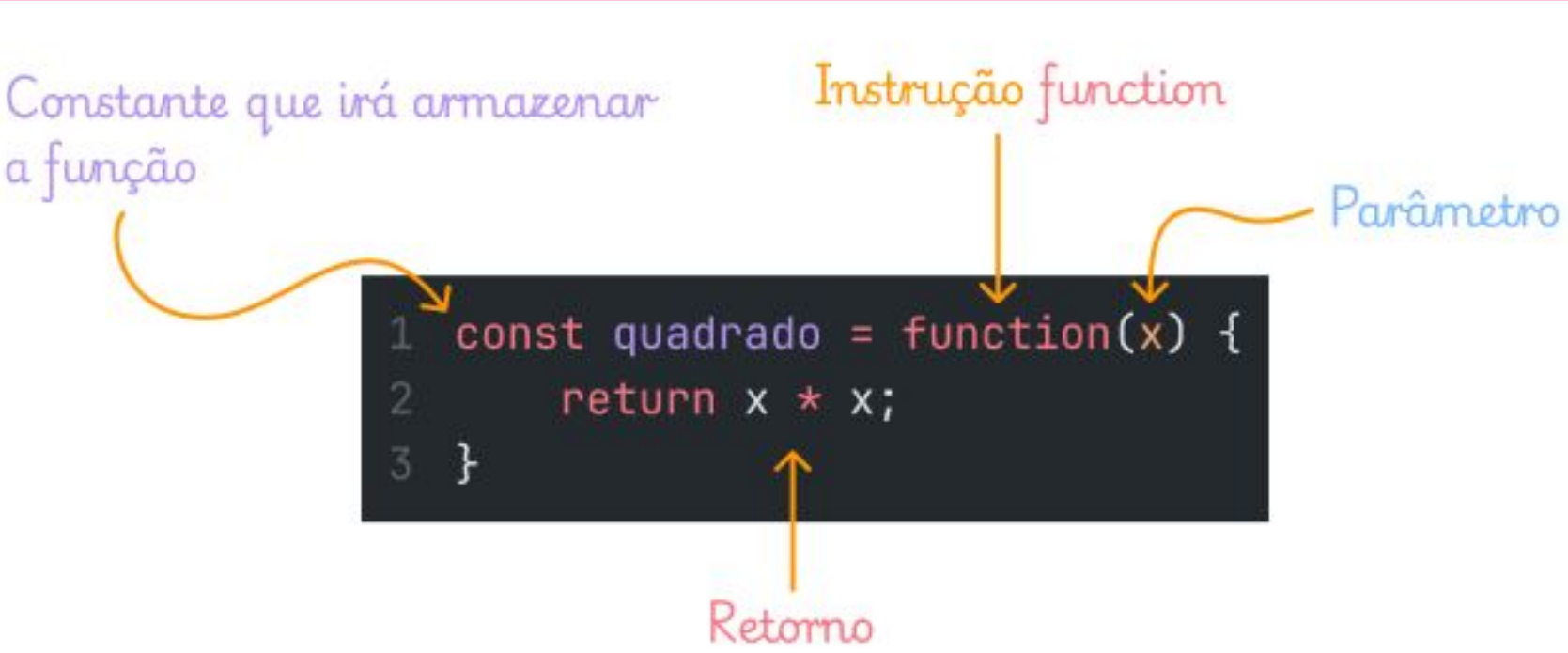
Uma expressão de função pode ser atribuída à uma **variável** ou **constante**.

O valor então associado a este identificador é uma **função**.

Por exemplo, este código define a constante quadrado para se referir a uma função que produz o quadrado de um determinado número.

```
const quadrado = function (x) {
  return x * x;
};

// Invocação da função quadrado passando o argumento 12
console.log(quadrado(12));
// Saída → 144
```



Estrutura de atribuição de uma expressão de função

Parâmetros

Uma função pode ter **vários parâmetros** ou **nenhum parâmetro**.

No exemplo a seguir, fazerBarulho não lista nenhum nome de parâmetro, enquanto soma (que faz a soma de dois números) declara dois parâmetros, **a** e **b**:

```
const fazerBarulho = function() {
  console.log("Vruuum!")
}

fazerBarulho()
// → Vruuum!

const soma = function(a, b) {
    let resultado = a + b
    return resultado
};

console.log(soma(23, 5))
// → 20
```

Declaração de função

function statement

Expressão de função

Uma função pode ser criada com uma **declaração** com a instrução **function.**

Um declaração function resultará em um objeto **Function** e tem todas as propriedades, métodos e comportamentos dos objetos **Function**.

Por padrão, funções retornam **undefined**. Para retornar qualquer outro valor, a função precisa ter uma instrução **return** que especifica o valor para retorno.

Declaração de função

Podemos declarar uma função utilizando a instrução **function** seguida de um nome

A instrução **return** no corpo da função especifica qual será o valor retornado pela função, se não especificado, ela retornará **undefined**.

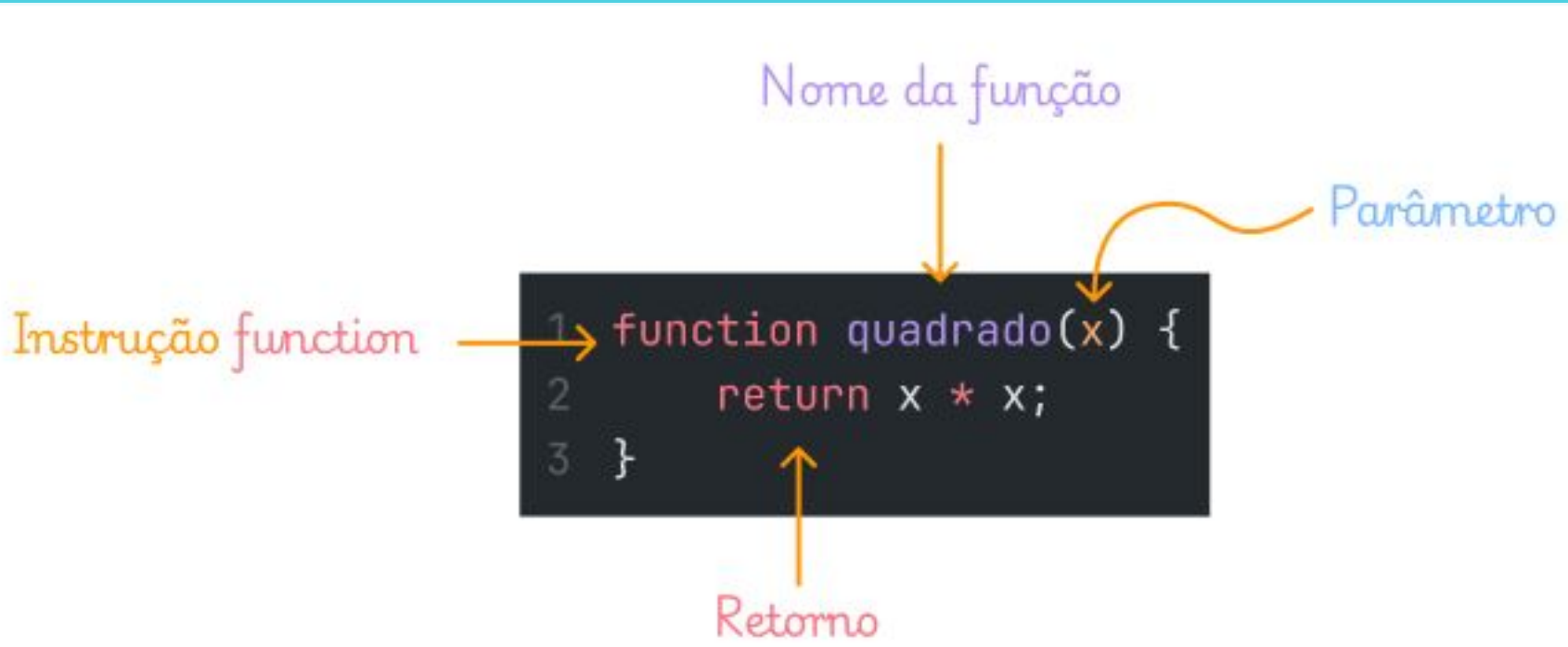
```
function quadrado(x) {
  return x * x;
}
```

Declaração de função

```
function areaDoRetangulo(ladoA, ladoB) {
    let area = ladoA * ladoB
    return area
}
// Invocação da função com diferentes parâmetros
let area1 = areaDoRetangulo(10, 10)
let area2 = areaDoRetangulo(2,4)
```

Declaração de função

```
function areaDoRetangulo(ladoA, ladoB) {
    let area = ladoA * ladoB
    return area
}
```



Estrutura de declaração de uma função

Içamento (*hoisting*)

O código funciona, mesmo que a função seja definida após o código que a invoca.

As declarações de função não fazem parte do fluxo de controle regular de cima para baixo.

Elas são conceitualmente movidas para o topo de seu escopo e podem ser utilizadas por todo o código nesse escopo. Isso às vezes é útil porque oferece a liberdade de ordenar o código da maneira que parecer mais clara, sem se preocupar em definir todas as funções antes de utilizá-las.

```
console.log(
    "Mensagem do futuro: ",
    mensagemDoFuturo()
)


function mensagemDoFuturo() {
  return "A IA ainda não capina lote";
}
```

Função seta

arrow function

Arrow function

Introduzidas em 2015 (ES6), as *arrow functions* oferecem uma sintaxe mais concisa usando o operador **=>** (daí seu nome).

A leitura do símbolo é mais ou menos "esta entrada produz este resultado".

```
const media = (n1, n2) => {
    return (n1 + n2) / 2
};
```

```
const media = (n1, n2) => {
    return (n1 + n2) / 2;
};
```

1

Parênteses opcionais

Com apenas **um parâmetro**, os parênteses podem ser omitidos.

```
const quadrado = x => return
x * x;
```

2

Retorno implícito

Se o corpo é uma **única expressão**, as chaves e o `return` são dispensáveis.

```
const quadrado = x => x * x;
```

3

Sem parâmetros

Use um par de parênteses vazio quando não houver parâmetros.

```
const latir = () =>
console.log("Roof!");
```

Closures

Conceito e aplicações

O que são closures?

Uma closure lhe dá acesso ao **escopo** de uma função externa a partir de uma função interna.

Em JavaScript, as closures são criadas toda vez que uma função é criada, no momento da criação da função.

Função aninhada

Exemplo de função aninhada com acesso à variável em escopo externo

```
function init() {
  var nome = "MeuApp"; // nome é uma variável local criada pelo init


  function exibirNome() {
    // displayName() é a função interna, uma closure
    console.log(nome); // usa a variável declarada na função pai
  }
  exibirNome(); // A invocação da função interna
}
init();
```

Função que retorna uma função

Lembrando que funções são objetos de primeira classe, que podem ser retornados por outras funções, podemos mudar um pouco nosso código, e ele ainda funcionará.

```
function init() {
  var nome = "MeuApp"; // nome é uma variável local criada pelo init
  function exibirNome() {
    // displayName() é a função interna, uma closure
    console.log(nome); // usa a variável declarada na função pai
  }
  return exibirNome;
}
let minhaFuncao = init(); // uma função exibirNome será armazenada em minhaFuncao
minhaFuncao();
```

Função que retorna uma função

Lembrando que funções são objetos de primeira classe, que podem ser retornados por outras funções, podemos mudar um pouco nosso código, e ele ainda funcionará.

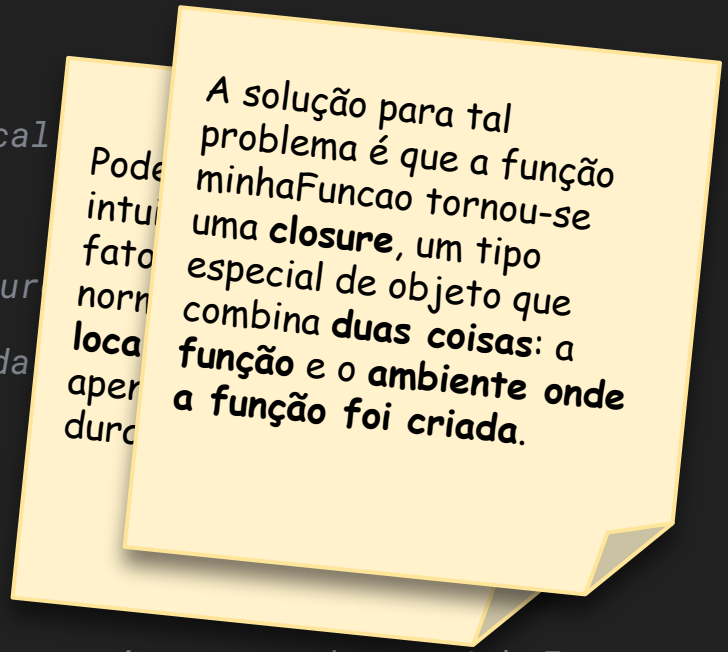
```
function init() {
  var nome = "MeuApp"; // nome é uma variável local
  function exibirNome() {
    // displayName() é a função interna, uma closur
    console.log(nome); // usa a variável declarada
  }
  return exibirNome;
}
let minhaFuncao = init(); // uma função exibirNome será armazenada em minhaFuncao
minhaFuncao();
```

Pode parecer não muito intuitivo de que o código de fato **funciona**, pois normalmente **variáveis locais** de uma **função**, apenas existem pela duração de sua execução.

Função que retorna uma função

Lembrando que funções são objetos de primeira classe, que podem ser retornados por outras funções, podemos mudar um pouco nosso código, e ele ainda funcionará.

```
function init() {
  var nome = "MeuApp"; // nome é uma variável local
  function exibirNome() {
    // displayName() é a função interna, uma closur
    console.log(nome); // usa a variável declarada
  }
  return exibirNome;
}
let minhaFuncao = init(); // uma função exibirNome será armazenada em minhaFuncao
minhaFuncao();
```



Escopo de variáveis

escopo global
escopo de função
escopo de bloco

Escopo

O escopo em JavaScript define a **visibilidade** e o **tempo de vida** de uma **variável**, ou seja, quais partes do seu código conseguem ler ou alterar essa variável.

Escopo

Escopo Global: variáveis declaradas fora de qualquer função ou bloco. Elas podem ser acessadas de qualquer lugar do código.

Escopo de Função: variáveis declaradas dentro de uma função. Elas só podem ser usadas dentro dessa função.

Escopo de Bloco: variáveis declaradas dentro de chaves {} (como em um if, for ou while). Elas só existem dentro daquele bloco

Escopo global e de função

Variáveis globais ficam acessíveis em todo o script, enquanto as de função não.

```
const globalVar = "Sou global";

function minhaFuncao() {
    var funcaoVar = "Sou local da função";
    console.log(globalVar);
    console.log(funcaoVar);
};

console.log(globalVar); // ✅ Funciona
console.log(funcaoVar); // ❌ Erro funcaoVar is not defined
```

O problema do var

O var não respeita blocos de código (como i**f** ou **for**), apenas funções. Veja como ele "vaza" de dentro de um bloco:

```
if (true) {
    var fruta = "Banana";
    let vegetal = "Cenoura";
}

console.log(fruta); // ✅ "Banana" (vazou do bloco)
console.log(vegetal); // ❌ vegetal is not defined
```

Casos de uso de Closures

Encapsulamento,
High Order Functions,
Currying

Encapsulamento de dados

A constante MAX, e as variáveis viagens e nomeDaMaquina, só estão disponíveis à própria closure.

```
function maquinaDoTempo(nome) {
  const MAX = 5;

  let viagens = 0;
  let nomeDaMaquina = nome

  return function viajar() {
    viagens++;

    if ( viagens <= MAX ) {
      console.log( `${nomeDaMaquina} Restam ${MAX - viagens} viagens...` );
    }
  }
}

let delorean = maquinaDoTempo('DeLorean');
let tardis = maquinaDoTempo('Tardis');

delorean();
delorean();

tardis();
```

High-order functions

A programação funcional promove o uso de funções de ordem superior, que são funções capazes de receber outras funções como argumentos ou de produzir funções como resultado.

As **closures** possibilitam a criação dessas funções de ordem superior, pois permitem que as funções "capturem" seu ambiente léxico e mantenham o acesso a variáveis do escopo externo, mesmo após a função pai ter concluído sua execução.

High-order function

Como funções são objetos, que podemos passar como parâmetros, podemos passar uma função que será usada por um função interna

```
function aplicarFuncaoEmArray(funcaoAplicar) {
  return function (valores) {
    const novosValores = [];
    for (const valor of valores) {
      novosValores.push(funcaoAplicar(valor));
    }
    return novosValores;
  }
}
const acrescentar10 = (x) => x + 10;
const numeros = [7, 13, 189, 25, 54, 0];
const numerosMais10 = aplicarFuncaoEmArray(acrescentar10)(numeros);
console.log(numerosMais10);
```

Currying

A técnica de Currying (nomeada a partir do matemático Haskell Curry), é a capacidade de converter uma função que recebe ***n*** argumentos em uma árvore de funções aninhadas, cada uma recebendo um único argumento.

A função retornada na invocação de uma função, é uma closure que "lembra" dos valores passados anteriormente, permitindo que apenas um argumento seja passado em sua invocação e a função mais interna seja finalmente executada.

Função que adiciona sufixo

```
function adicionaSufixo(sufixo_a_adicionar) {
  let sufixo = sufixo_a_adicionar;
  return function(termo) {
    console.log( termo + sufixo );
  }
}

let email = adicionaSufixo("@email.com.br");
email("joana");
email("osmar");
```

Função que adiciona sufixo

```
function adicionaSufixo(sufixo_a_adicionar) {
  let sufixo = sufixo_a_adicionar;

  return function(termo) {
    console.log( termo + sufixo );
  }
}

let email = adicionaSufixo("@email.com.br");
let emailEdu = adicionaSufixo("@edu.email.com.br");
email("joana");
emailEdu("joana");
```

Função que adiciona prefixo e sufixo

```
function adicionaPrefixoSufixo(sufixo) {
  return function (prefixo) {
    return function (termo) {
      console.log( prefixo + termo + sufixo );
    }
  }
}

let email =
adicionaPrefixoSufixo("@edu.email.com.br")("empresa_")("joana");
```

Módulos

`import, export, default`

Módulos

Os programas JavaScript começaram muito pequenos - a maior parte do seu uso nos primeiros dias era para executar tarefas isoladas.

Com o avanço da tecnologia agora temos aplicativos completos sendo executados em navegadores com muito JavaScript, além de JavaScript ser usado em outros contextos (Node.js, por exemplo).

Módulos

O **Node.js** possui essa capacidade há muito tempo e existem várias bibliotecas e estruturas JavaScript que permitem o uso do módulo (por exemplo, outros **CommonJS** e **AMD** de módulos baseados em **RequireJS**, e mais recentemente **Webpack** e **Babel**).

Módulos

O **Node.js** possui essa capacidade há muito tempo e existem várias bibliotecas e estruturas JavaScript que permitem o uso do módulo (por exemplo, outros **CommonJS** e **AMD** de módulos baseados em **RequireJS**, e mais recentemente **Webpack** e **Babel**).

A boa notícia é que (finalmente) os navegadores modernos começaram a dar suporte nativamente à funcionalidade do módulo

Export

```
export const nome = "defesa_civil";

export function alerta(nivel) {
    if (nivel === 'severo') {
        alert("Alerta Severo! Chuvas fortes nas próximas horas.");
        return;
    }

    if ( nivel === 'extremo') {
        alert("Alerta Extremo! Aja imediatamente, abrigue-se!");
    }
}
```

A primeira coisa que você faz para obter acesso aos recursos do módulo é exportá-los.

A maneira mais fácil de usá-lo é colocá-lo na frente de qualquer item que você queira exportar para fora do módulo, por exemplo:

Você pode exportar **var**, **let**, **const**, **functions** e **class**.

Export

Uma maneira mais conveniente de exportar todos os itens que você deseja exportar é usar uma única instrução de exportação no final do arquivo do módulo, seguida por uma lista separada por vírgula dos recursos que você deseja exportar envoltos em chaves. Por exemplo:

```
export { name, draw, reportArea, reportPerimeter };
```

Import

Depois de exportar alguns recursos do seu módulo, é necessário importá-los para o script para poder usá-los. A maneira mais simples de fazer isso é a seguinte:

```
import { name, draw, reportArea, reportPerimeter } from
"./modules/square.js";
```

Você usa o **import** , seguida por uma lista separada por vírgulas dos recursos que você deseja importar agrupados em chaves, seguidos pela palavra-chave **from**, seguida pelo caminho para o arquivo do módulo.

Aplicando o módulo ao seu HTML

Primeiro de tudo, você precisa incluir **type="module"** no **<script>** elemento, para declarar esse script como um módulo. (no script principal, o qual importará outros módulos, os demais não serão incluídos via HTML, mas sim via import, esse é nosso intuito)

```
<script type="module" src="main.js"></script>
```

O script para o qual você importa os recursos do módulo atua basicamente como o módulo de nível superior.

Você só pode usar import e export instruções dentro de módulos, não scripts regulares.

Import

```
import { name, draw, reportArea, reportPerimeter } from
"./modules/square";
```

Nota: em alguns sistemas de módulos, você pode omitir a extensão do arquivo e o ponto(e.g. '/modules/square'). Isso **não funciona** nos módulos JavaScript **nativos**.

Diferenças entre módulos e scripts padrão

Não funcionam localmente: Atenção nos testes locais: se você tentar carregar o arquivo HTML localmente (i.e. com um arquivo ://), você encontrará erros do CORS devido a requisitos de segurança do módulo JavaScript. **Você precisa fazer seus testes através de um servidor**.

Modo strict: além disso, observe que você pode obter um comportamento diferente das seções de script definidas dentro dos módulos e não nos scripts padrão. Isso ocorre porque os módulos usam strict mode automaticamente.

Execução única: os módulos são executados apenas uma vez, mesmo que tenham sido referenciados em várias tags <script>.

Export default

Há também um tipo de exportação chamado default export — isso foi projetado para facilitar a função padrão fornecida por um módulo e também ajuda os módulos JavaScript a interoperar com os sistemas de módulos CommonJS e AMD existentes.

```
export default randomGeneratorFunc;
```

Renomeando importações e exportações

Dentro dos colchetes da instrução de importação e exportação, você pode usar a palavra-chave junto com um novo nome de recurso, para alterar o nome de identificação que será usado para um recurso dentro do módulo de nível superior.

```
export { funcaoA as funcaoLocalA } from "./modules/mo.js";
```

Referências Bibliográficas

MDN - Mozilla Developer Network. **Closures**. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Closures.

MDN - Mozilla Developer Network. **Módulos**. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Guide/Modules.

SARASWATHI, M A. **Closures in Action: Practical Use Cases**. Medium. Disponível em: https://javascript.plainenglish.io/closures-in-action-practical-use-cases-12b26509a068.

DE MEULDER, Olivier. **I never understood JavaScript closures**. Medium. Disponível em: https://medium.com/dailyjs/i-never-understood-javascript-closures-9663703368e8